

Convolutional Neural Network Accelerator for Handwritten Number Recognition on FPGA

Justin Hsu^{#1}, Daniel Narevich^{*2}

[#]*University of California, Davis*
Department of Electrical Engineering

¹jchhsu@ucdavis.edu

^{*}*University of California, Davis*
Department of Electrical Engineering

²djnarevich@ucdavis.edu

Abstract— Convolutional Neural Nets (CNNs) are a machine learning model capable of extracting specific features from an input-making them especially capable in tasks like image recognition. Recognizing handwritten digits is an especially important task in areas like banking, for digitizing and processing handwritten checks. Utilizing a GPU for such a model is inefficient as such a model cannot fully utilize a GPU for inference, thus consuming more power and needing more cost than necessary. As such, designing a custom accelerator for this model on FPGA is critical for both power consumption and cost. The dynamic hardware of an FPGA allows for the optimization of the accelerator-ensuring full utilization of the hardware. This project explores the differences in performance, power, and area across four different stationary architectures for systolic arrays: input stationary, weight stationary, output stationary, and row stationary.

Keywords— Convolutional Neural Network, Accelerator, Systolic Array, Kernel, Multiply-Accumulate, Processing Element, FPGA, Image Processing

I. INTRODUCTION

A convolutional neural network (CNN) is a machine learning model that uses the convolution operation to determine the similarity between an input and a kernel. The kernel is a filter matrix that is overlaid across the entire input and produces magnitudes for each overlay that represent the similarity between the input and what the kernel is filtering for. This CNN extracts specific features of handwritten numbers and provides a prediction of what that number is. Our CNN is built on the LeNet-5 architecture [1], but with our own trained weights. This model is then applied to a hardware accelerator we designed and implemented on a DE1-SoC FPGA using System Verilog. A hardware accelerator is a domain specific processor that is tailored to a specific model. The accelerator is designed so the model can fully utilize the hardware and perform specific operations much more

efficiently than a standard processor. Since a standard processor trades high generalization for low performance, many operations of that processor for an operation like convolution are spent on data management. The processor must cycle through the indexes of each input to the convolution, and manage the index of the output to ensure the calculation is stored in the proper area. As a result, less than 10% of all instructions issued are dedicated to the multiply-accumulate (MAC) operation that is the crux of convolution, while the remaining 90% of instructions are for data and memory management [2]. Because hardware accelerators are built for specific models, the hardware can be designed in such a way that takes advantage of the data layout, and the MAC operations take up the majority of the processing instead as they are the critical operation to CNNs. This hardware accelerator is implemented on the Terasic DE1-SoC [3] FPGA. The dynamic hardware reconfigurability of an FPGA means twofold: first, hardware can be easily instanced or removed in parallel to increase throughput or to meet power requirements, and second, hardware design can be easily changed should errors arise. This flexibility allows for quick and easy prototyping as our design is not permanently etched into silicon, and the overall architecture and number of functional units can be optimized to specifically target 100% hardware utilization by the CNN model.

II. CNN MODEL DEVELOPMENT

While this CNN was built using the same architecture and dataset as LeNet-5, the weights were computed using our own training. Using an

RTX 5050 GPU and the PyTorch machine learning framework, we developed a script in Python to train this CNN. Using the MNIST (Modified National Institute of Standards and Technology) dataset [4], a collection of thousands of 28x28 grayscale images of handwritten numbers (0 through 9), our model was trained.

A. Quantization

Quantization represents the shrinking of data within a machine learning model to save memory space. This could be represented by converting from floating point to fixed point numbers, or by shrinking the number of bits used to represent that data. One major concern of this project is the available amount of block RAM (BRAM) available onboard the FPGA to store the inputs, weights, and outputs. As such, a quantization scheme was chosen to balance the reduction in storage capacity needed for all the weights while maintaining high model accuracy. The weights were chosen to be quantized to int4-this way we can effectively halve the amount of storage needed for the weights (compared to int8), while barely suffering a drop to model accuracy. Additionally, our training used a quantization aware training (QAT) method [4] by introducing and accounting for quantization during training. The resultant trained model is not only more accurate even after aggressive quantization schemes compared to non-QAT trained models with the same quantization scheme applied, but also more robust against errors. As a result, our training developed a CNN model capable of predicting handwritten numbers from the MNIST dataset with >98% accuracy as shown in figure 1.

```
jhsu2022@Justins-MacBook-Air-2 EEC-289Q % python3 FPGA_4_bit.py
Starting Hardware-Optimized QAT on mps...
Epoch 1 | Loss: 0.3337 | Train Acc: 96.22%
Epoch 2 | Loss: 0.0875 | Train Acc: 98.19%
Epoch 3 | Loss: 0.0635 | Train Acc: 98.39%
Epoch 4 | Loss: 0.0496 | Train Acc: 98.86%
Epoch 5 | Loss: 0.0420 | Train Acc: 98.94%

--- Final Performance ---
Inference Accuracy (Test Set): 98.83%

Exporting INT4 weights and shift factors...
Layer: conv1.weight | Bit-shift (n): -4
Layer: conv2.weight | Bit-shift (n): -4
Layer: fc1.weight | Bit-shift (n): -4
Layer: fc2.weight | Bit-shift (n): -4
Layer: fc3.weight | Bit-shift (n): -4

Done! Weights saved to 'fpga_weights_int4/' directory.
```

Fig. 1 Training results of our model

A set of weights were produced after training concluded, exported to .csv format for ease of translation to fit onto the hardware.

III. HARDWARE ACCELERATOR ARCHITECTURE

Following the LeNet-5 architecture, our hardware accelerator architecture is shown in figure 2.

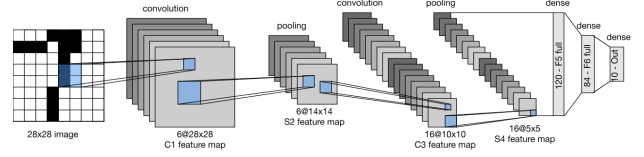


Fig. 2 Architecture of LeNet-5 model

A. Convolutional Layers

The convolutional layers are integral to the CNN, as they perform the bulk of the necessary computations needed for accurate prediction. As such, a computationally performant, efficient structure for convolution is necessary. Due to this, a systolic array architecture was chosen. Systolic arrays are large structures of processing elements (PEs) in a 2D grid. As inputs enter through the top and left edges, each PE performs a MAC operation and passes the data to the downstream adjacent PE. This parallel structure takes advantage of data reuse to drastically improve the performance of systolic arrays compared to general performance processors like CPUs and GPUs. Not only do CPUs and GPUs have instruction overhead for data management, but their performance is massively limited by the performance of the memory. Due to the memory wall of processors, the latency and bandwidth of memory has improved at such a decreased rate compared to the throughput of processors that the memory can no longer sufficiently provide enough data for the processor, so the majority of time is spent waiting for data to be transferred rather than waiting for computation [5]. Systolic arrays allow for high levels of data reuse, and take advantage of the constant data reuse in a convolution operation-the kernel data is constantly reused, while the input data is reused across different kernels. By streaming and replicating the data instead of constantly re-reading data from memory, the

systolic array can amortize the latency of DRAM accesses by only needing to access a piece of data once for a layer. Similarly, by utilizing registers within the PEs to handle accumulation, a systolic array can avoid constant writes to memory-as it only has to write after the full partial sum has been completed.

The systolic array was designed using a 2D grid of PEs. Each PE has a multiplier, an adder, and a register. It takes the input activation and weight, multiplies them together, and adds them to whatever value is held in the register already. To account for the ever increasing size of accumulated data, the register has width of $2N + \log_2(M)$ [6], where N is the width of the inputs and M is the number of additions required. Any value within the PE (the input activation, the weight, and the output) can be streamed directly to the output of the PE, as this way data can be streamed to downstream adjacent PEs, making use of data replication. The remaining value is held within the PE, to be reused within the individual PE. The PEs are then arranged in a grid pattern, where the input to each PE is the output of the upstream PE.

Since systolic arrays require data to be streamed into a PE for an output to be generated, this means that data must be skewed to ensure the correct two sets of input data meet within a PE to generate the correct output. Each subsequent row must introduce a one cycle input delay compared to the upstream row, and each subsequent column must also introduce a one cycle input delay compared to the upstream column. To manage the data skewing, FIFOs are used and connected to each input in the row and column. Each FIFO has a read enable port that is activated with each subsequent clock cycle, ensuring correct skewing of the input data. To manage data de-skewing to account for the offset output generation, a delay pipe is used by connecting several shift registers together. Outputs that are generated first enter the pipe and are shifted downstream, while outputs that are generated later enter the delay pipe later. This flattens the outputs into a 1D vector so it can be written into memory.

To achieve the highest amount of performance, a very large number of PEs must be instanced so each input row can be calculated simultaneously through parallel functional blocks. However, this is the best

case scenario with an infinite number of hardware resources available. Obviously, this is an unrealistic expectation, especially given the stringent hardware constraints of the DE1-SoC. As such, tiling must be implemented. Since there is not enough hardware to process the entire input simultaneously, if blocks of the input were instead mapped to parallel instances of the systolic array, and these parallel systolic arrays were reused for different blocks at a constant offset, this hardware can be reused multiple times to calculate the entire input, rather than waiting for the inputs to be calculated sequentially. This means the systolic array tiling control can act as a nested loop in programming-use all the available parallel arrays to calculate so many values simultaneously then reuse this hardware block so many number of times to finish the entire input. By using tiling, very large input activation matrices can be processed without needing a massive hardware array through hardware reuse. However, the downside is that tiling requires significant control logic to partition the data correctly into the hardware array. Furthermore, if parallel systolic arrays are used, more physical area will be required for implementation-a concern for this project due to the limited resources of the FPGA.

The architecture of the systolic array can be changed, as all data values a PE produces and operates on can be designed to be streamed through and sent to adjacent PEs or held stationary.

1) *Weight Stationary*: Weight stationary architecture involves the input activation and outputs to be streamed to downstream PEs, and weights to be held within the PE for reuse. This type of architecture is very efficient for models requiring many batches-many input activations using one weight kernel-as the weight remains stationary within and does not have to be constantly pulled from memory to be reused. Instead, the input activations are streamed through, which is highly optimal for many batch models as the weights never have to be reloaded from memory. This architecture can be used in high batch or high throughput inference where the same model is used to process inputs. Conversely, this architecture struggles with models that use a small amount of batches, as weights have to be constantly reloaded from memory. Furthermore, this can introduce a memory bottleneck when pulling the weights from memory, as there might not be sufficient bandwidth to account for every weight being loaded at once.

2) *Input Stationary*: Input stationary architecture is incredibly efficient for models that require many channels for

feature extraction. Inputs remain within individual PEs to be reused, while weights and outputs are streamed through. Since output channels are created from one input activation used across multiple weight kernels, this type of architecture excels at handling these types of many-channel models. This type of architecture can be utilized for edge AI devices, where applications typically have a batch size of 1, but require many channels for extraction. However, this presents two issues: firstly, this type of architecture is inefficient for large batch processing, and secondly, distributing the stationary inputs across the array simultaneously can require complex data broadcasting hardware as the input activation memory has very high fan-out.

3) *Output Stationary*: Output stationary architecture is highly adaptable to input activation and weight kernel matrix dimensions as each individual PE acts merely as an accumulator. By streaming both the input activations and weights, while maintaining the output within the PE, this type of architecture can provide superior performance when high precision outputs are produced since the outputs do not move, thus saving switching energy and significant wiring overhead. This type of architecture can be highly efficient in implementing fully connected layers where minimizing the movement of high precision accumulated data is the highest priority. However, high bandwidth is required to support the constantly streaming input activation and weights, doubling the required routing resources and register file reads. Furthermore, there is high drain overhead, as these wide output values must all be cleared before the next set of data can be processed.

4) *Row Stationary*: Row stationary architecture was popularized by the MIT Eyeriss accelerator [7]. By breaking down 2D convolutions into a 1D row, it maximizes all three forms of reuse as the input activations, weights, and outputs all within a localized cluster of PEs. The weights are held statically within a PE row, while inputs flow diagonally upwards and to the right, while outputs are passed vertically downwards. By keeping data movement within the neighboring PEs, this architecture minimizes the amount of memory accesses required, enjoying high energy efficiency. This type of architecture can be used in citations where heavy data reuse (from sliding windows) can be heavily exploited. However, there are two major drawbacks: firstly, the control logic is very complex, as the PEs require sophisticated control systems to handle the multi-directional dataflow. Secondly, there is a compiler dependency as it requires a complicated mapping tool to optimally partition the workloads into rows.

B. Pooling Layers

The pooling layers reduce the size of layers deeper in the model, reducing the required computations for both training and inference. This reduction in complexity also helps reduce the risk of overfitting in the model. While not traditionally the task of a pooling layer, our pooling layer also

requantizes the output values which are produced by the prior convolution layer. Our pooling layer uses average pooling to calculate the output value of the combined pixel. We chose average pooling as it maintains the effect of all of the input pixel values which we hope will improve the overall accuracy of the model. It is also the architecture of the original LeNet, though many newer versions use max pooling. Average pooling, as the name implies, works by calculating the mean of all input numbers. Our architecture uses a two by two window of input values with a stride of two, covering each input pixel exactly once. We implemented this pooling layer in SystemVerilog. The core of the pooling layer is a module which takes in four inputs, sums them, and uses bit shifting to quantize the value. The value is only quantized if it lies within the possible range for the integer quantized value. If it happens to be larger, it is set to the largest possible quantized value. This is a simple way to eliminate outliers. This decreases the risk of overfitting and allows us to more accurately represent smaller values, which include most of the expected outputs from our pooling process. In our final architecture, a single pooling module is instantiated, which processes one two by two window at a time. The data from the prior layer is fed from the output BRAM one window at a time, and the output is also fed into BRAM to be read by the next layer. Our architecture has two pooling layers, one after each convolution layer. Both reduce the number of pixels by a quarter, allowing for lower computational intensity in both the second convolution layer and the fully connected layers. In hardware, the pooling layers take up a minimal percentage of the hardware usage thanks to their simple computation. If we used a larger systolic array, parallelising the pooling hardware could increase the processing speed of the model, but in the current architecture the pooling layer is appropriately sized and does not cause delay and takes up the minimum possible physical area.

C. Fully Connected Layers

The fully connected layers contain elements that are each connected to all neurons in the previous layer. The primary purpose of the fully connected layer is to take the more complex features of the

prior layers, which may detect specific structures in small areas of the image, and combine them to create more broad features, eventually combining them into ten final features which represent the digits zero through nine. Since each neuron is connected to all other neurons in the next layer, the fully connected layers can take up a large portion of the FPGA’s routing capacity, and take up a significant portion of hardware, especially if the systolic arrays are highly hardware efficient. The further down a neuron is in the fully connected pipeline the more of the model it is exposed to, resulting in broader features. Because there are so many connections between neurons, many weights and biases are needed which can result in high memory demands for the fully connected layers. The first and largest fully connected layer requires by far the most memory of any layer, with 48000 four bit weights. In our architecture, there are three fully connected layers, which each reduce the number of features. Much like the systolic array, the core of the fully connected layer is a MAC. This System Verilog MAC module is used to define the layer and takes the previous layer’s inputs, and defines an array of values that can be plugged into the next fully connected layer, creating a pipeline. To maintain the pipeline, a serializer is placed in between each layer, which allows the layers to process smaller batches of data, instead of running large calculations at once which would require an excessive amount of pins and hardware area. The fully connected stack will then output a one hot encoded output which defines the final result, which can also be displayed on a seven segment display on the DE1-SoC evaluation board.

IV. RESULTS

The overall architecture hardware resource usage is summarized in Table 1. Each architecture has the same amount of memory writes because the same number of outputs are generated, and thus must be written to memory. The hardware is capable of detecting all number values from 0-9. The prediction accuracy across all architectures is very high-in a test of 20 inputs (using the same test set across all architectures), the hardware correctly predicts 19 of them, giving an accuracy of 95%.

The singular misprediction can be attributed to two possible causes: first, the model is predicting wrong. Our trained model using PyTorch had a prediction accuracy of $\sim 98.5\%$, so while the chances of the model predicting incorrectly is possible, the high model accuracy suggests this is unlikely. The second, likelier option is the input itself is very messy. Since all the inputs are pulled from the MNIST dataset-a dataset of *handwritten* numbers-the input image number could be so messy that the model cannot tell what number the input is, thus predicting wrong. This is far more likely, as even humans struggle to decipher messy handwriting sometimes. For example, a very messy 1 could be perceived as a 7, or a 4 as a 9, and so on. The ambiguous nature of messy handwriting prevents accurate predictions, something that is true regardless of whether the predictor is a human or a machine learning model.

```
=====
BATCH SIMULATION COMPLETE
Passed: 19 / 20
Failed: 1 / 20
OVERALL: 1 MISMATCH(ES) - check if only one digit works
=====
```

Fig. 3 Simulation results of our model

```
-----
RUN 19 / 20 | MNIST test index 118
-----
[TB INFO] Successfully extracted and padded MNIST Image Index 118.
[TB INFO] Backdoor load to conv1 input feature memory (i_mem) complete.
[TB INFO] Successfully extracted ground truth label: 9
[TB INFO] Asserting start for expected label 9...
[TB INFO] Pipeline active. Awaiting done...
Hardware predicted output: 4
RUN 19 FAIL | index 118 | expected 9 (one-hot 1000000000) | LED 0000010000
```

Fig. 4 Incorrect prediction by hardware

TABLE I
RESOURCE UTILIZATION ACROSS DIFFERENT SYSTOLIC ARCHITECTURES

Architecture	Compute Time (ms)	Memory Accesses	Power (mW)
Input Stationary	20.955	2000 writes 98000 reads	548.21
Weight Stationary	8.906	2000 writes 125 reads	613.38
Output Stationary	8.8453	2000 writes 3920 reads	546.59
Row Stationary	13.947	2000 writes 75000 reads	622.08
Architecture	Logic Utilization (%)	Registers Used	Block Memory Bits
Input Stationary	23	12240	962,880
Weight Stationary	29	20786	1,679,680

Output Stationary	20	10580	862,528
Row Stationary	39	21796	868,672
Architecture	Logic LABs Utilization (%)		
Input Stationary	40		
Weight Stationary	56		
Output Stationary	36		
Row Stationary	80		

1) *Input Stationary*: Input stationary architecture has the lowest throughput, the highest overall memory traffic from its 98,000 reads. This is because the batch size of this model is 1, thus input stationary cannot be taken full advantage of. This model uses the same weights across different inputs, rather than different weights across the same inputs, leading to such a high amount of memory reads. The weight values are discarded after being used once by an input, resulting in weights having to be read from memory for every input, leading to such a high amount of read accesses. The sheer amount of memory reads causes throughput to plummet significantly, as it is more than half as slow as the fastest architectures. This is due to the hardware sitting idle, waiting for data to be read from the memory so the hardware has data to be processed on. The idling hardware explains the power paradox-even though memory traffic is high, overall power consumption remains relatively low because a lot of hardware is sitting idle instead of doing work. This means lower dynamic switching activity, which keeps the power low despite poor performance.

2) *Weight Stationary*: Overall, the weight stationary architecture performed the best in terms of throughput, and memory accesses, but uses the second most amount of logic resources and registers, and uses by far the most amount of block memory. The memory reads are the lowest because the filter kernels are constantly reused, and in this model with a very small batch size (batch size = 1), the weights remaining within the PEs can be taken advantage of as there is no need to swap filter kernels at all. However, in order to maintain weights within the PEs, the BRAM usage is almost double all the other architectures. This is due to the fact that the weights for the network layers must be heavily partitioned across the FPGA so multiple PEs can read them simultaneously without port conflict. This is how the amount of memory reads is so low, since there is very high reuse of the weight values. Furthermore, this is how throughput is so high, because instead of waiting for weight values to be read from memory, they stay within the PEs. Thus, processing can always happen and hardware does not sit idle waiting for data to be read from memory.

3) *Output Stationary*: Output stationary has the highest throughput, is the most power efficient, and the lowest amount of logic utilization. Memory traffic is relatively low with 2000

writes and 3920 reads because the partial sums stay within the PEs instead of being written to memory and then read back. This results in the highest throughput by minimizing the amount of hardware idle time since the wide output results do not have to be constantly written to and read from memory. Furthermore, the logic utilization is the lowest because the PEs are acting as simple registers, rather than a complex array of weights or row buffers. The routing resources are relaxed significantly due to this, and the low utilization is reflected in these results. Power is also the lowest because of this, since large partial sums do not need to be transferred constantly, minimizing the dynamic switching and reducing power consumption.

4) *Row Stationary*: Row stationary architecture has the highest power consumption, highest amount of hardware utilization, and a relatively high amount of memory accesses. Row stationary attempts to maximize reuse of all data types by mapping 1D rows of convolutions to rows of PEs. However, in a model as small as ours, and with the layout of the model data, row stationary architecture is unable to take advantage of the reuse of all data types. The shallow layers of our model combined with line buffer usage for control means rows end up getting re-fetched repeatedly from memory. Furthermore, the complexity of row stationary control logic significantly increases control logic overhead-using valuable routing and logic resources. As such, 39% of logic resources were used, and the most amount of registers were used. This explains the high power consumption, as the high resource utilization and register usage contributes to high dynamic switching, resulting in the highest power consumption.

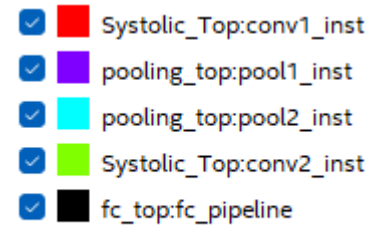


Fig. 5 Layout key for figure 6-8

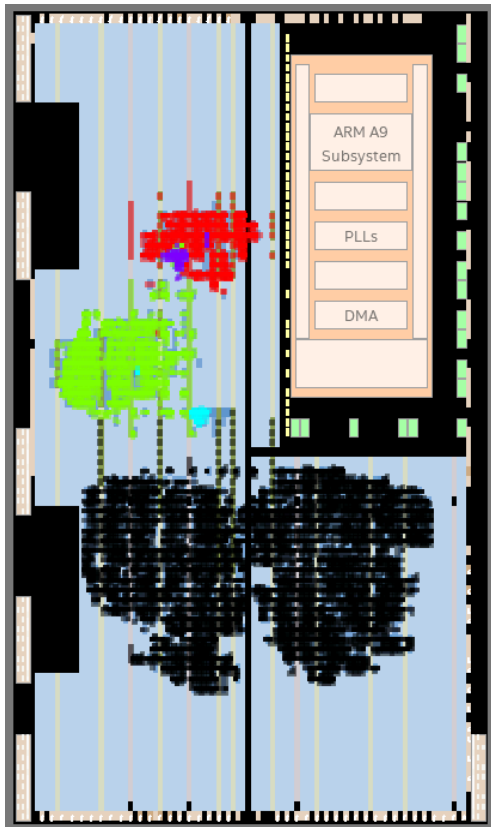


Fig. 6 Layout of input stationary architecture

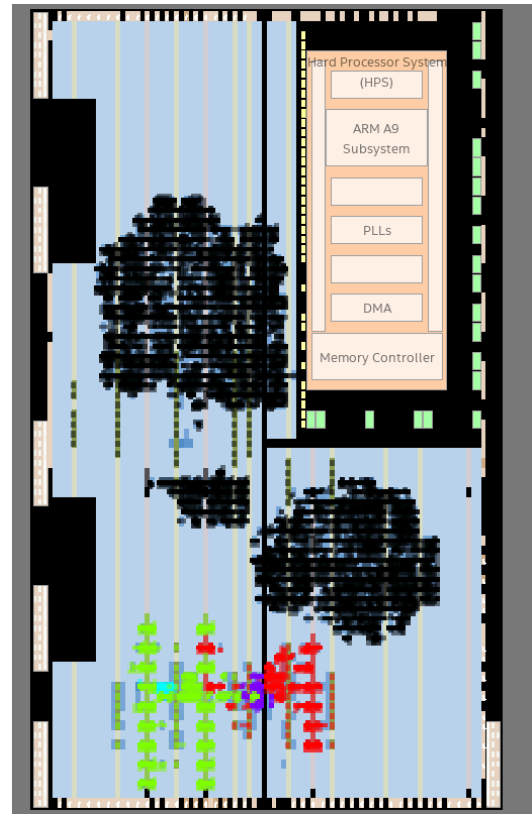


Fig. 8 Layout of output stationary architecture

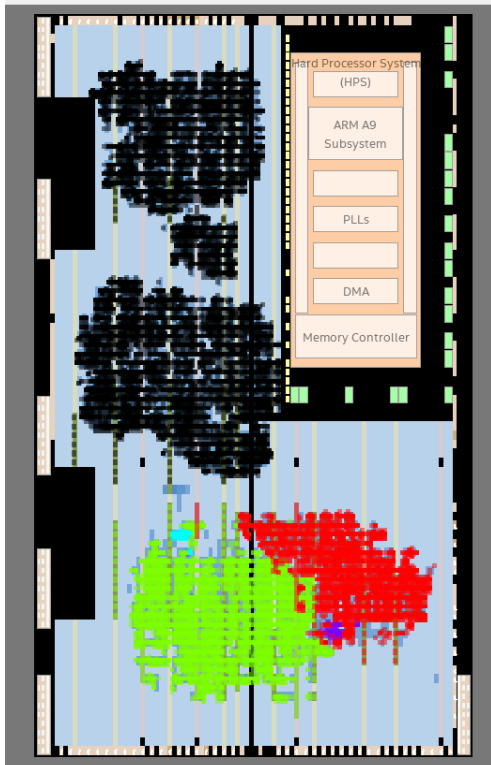


Fig. 7 Layout of weight stationary architecture

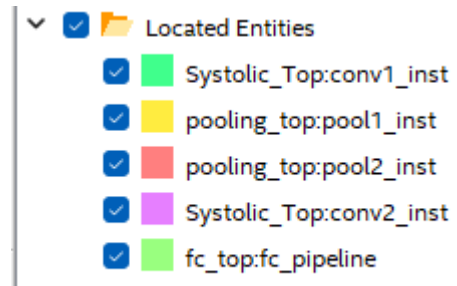


Fig. 9 Layout key for figure 10

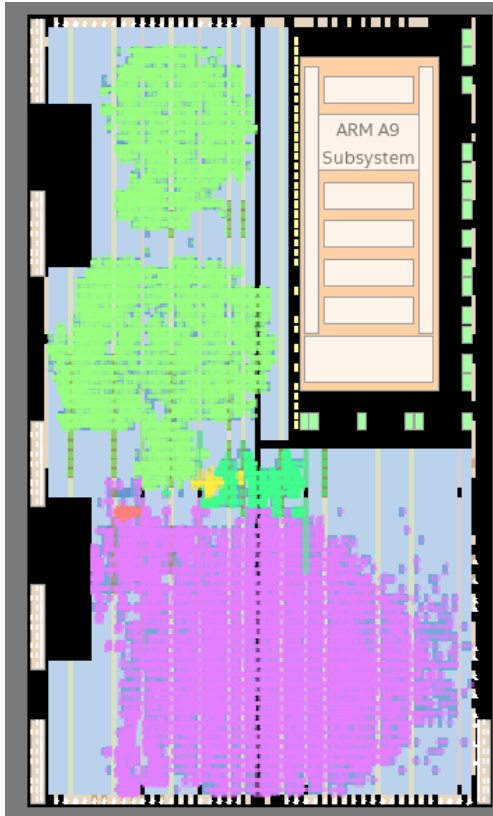


Fig. 10 Layout of row stationary architecture

V. DISCUSSIONS

Through our various architectural implementations of a hardware accelerator for handwritten number recognition on FPGA, we concluded that the two best architectures for this specific model are weight stationary and output stationary. This is due to their superior throughputs, and relatively low memory traffic. Although both architectures have drawbacks when it comes to the physical area of implementation, their performance benefits outweigh the area penalty because this implementation is specifically deployed on an FPGA. This means that the physical array itself could be instantiated with extra PEs, or more physical arrays could be instantiated to benefit from parallel hardware reuse. On the other hand, even though the available FPGA hardware was underutilized, this consequence is not as detrimental compared to an ASIC underutilizing a piece of silicon. Because FPGAs can be quickly repurposed into any circuit, this means redeploying a design with more functional units than before to maximize

resource utilization is much easier, unlike a monolithic piece of silicon that cannot be changed once it is etched. As a result, underutilizing physical area on an ASIC design is much more costly than underutilizing an area on an FPGA design.

Although we have concluded that weight stationary and row stationary architectures are the best for this model, this conclusion does not hold true in general. This result and conclusion holds true only for this specific model. Because this model has a batch size of 1, and is very small and shallow compared to other machine learning models, it heavily benefitted from weight stationary and output stationary architecture. If a different model was used that had very large batch sizes instead, it is possible that weight stationary might not be the optimal architecture, but rather input stationary. Furthermore, if a very deep and large model were used instead, it is very likely that different architectures such as row stationary could be the best design due to being able to take advantage of significant amounts of data reuse. All this to say that choice of systolic array architecture is not a static decision. There is no one best overall architecture, as architecture requires context of the model in terms of how data is laid out, the layers of the model itself, etc.

Additionally, while throughput was prioritized for this project, other metrics are important to consider as well. When designing any system, power, performance, and area (PPA) analysis must be conducted. The choice of which architecture may also depend on what factor to prioritize. Certain systems may sacrifice throughput for physical area minimization and lower power consumption in the case of edge devices and battery powered devices. Different systems may prioritize different metrics, thus choose alternative architectures.

VI. CONCLUSION

In conclusion, through implementation of various systolic array architectures on FPGA, we have designed a hardware accelerator capable of inferring handwritten numbers from images. The hardware accelerator is capable of predicting handwritten numbers with >95% accuracy, and only struggles when the input image is incredibly

messy-something even humans struggle to comprehend. Overall, this project was successful in its goals of designing a hardware accelerator with four different systolic array architectures, and through analysis of the results we are able to conclude that for this specific machine learning model, weight stationary and output stationary systolic array architecture offers the best balance between power, performance, and area.

REFERENCES

- [1] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2018.
- [2] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and Memory Wall," *IEEE Micro*, pp. 1–5, Jan. 2024, doi: <https://doi.org/10.1109/mm.2024.3373763>.
- [3] Terasic Technologies, "Terasic - DE Boards - Cyclone - DE1-SoC Development and Education Kit," *Terasic.com.tw*, 2026. <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=165&No=836#contents> (accessed Jun. 11, 2026).
- [4] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *arXiv (Cornell University)*, Dec. 2017, doi: <https://doi.org/10.48550/arxiv.1712.05877>.
- [5] K. Roy *et al.*, "Breaking the memory wall: next-generation artificial intelligence hardware," *Frontiers in Science*, vol. 3, Dec. 2025, doi: <https://doi.org/10.3389/fsci.2025.1611658>.
- [6] A. Sasan, "Quantization," Class Lecture, EEC 289Q.
- [7] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks," *IEEE Xplore*, Jun. 01, 2016. <https://ieeexplore.ieee.org/document/755140>
- [8] L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]," in *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, Nov. 2012, doi: [10.1109/MSP.2012.2211477](https://doi.org/10.1109/MSP.2012.2211477).